

LAB 03 – BUILDING YOUR FIRST PLC PROGRAM

Tank Fill Control Using OpenPLC

Name: Adamu Nando Usufa
Registration Number: C11/26/ACIS/17367
Course: Intro to Critical Infrastructure for IT Learners
Lab Number: Lab 03
Date: 28 September 2026

1 Project Configuration

The TankFill PLC project was created in OpenPLC Editor using Ladder Diagram (LD).

Table 1: Project configuration

Parameter	Configuration
Virtual Machine	ICDFA-PLC-Engineering-v1.0
Snapshot	LAB03-START
Project	TankFill
POU	TankFill
Language	Ladder Diagram (LD)
PLC Address	192.168.1.10

2 Variables and Setpoints

The required PLC variables were configured using BOOL for discrete control states and REAL for tank-level measurements and setpoints.

Table 2: Configured PLC variables

Variable	Type	Value
TankFillAuto_Cmd	BOOL	–
TankValveOpen	BOOL	–
Estop_Cmd	BOOL	–
TankLevelHi_Alarm	BOOL	–
TankLevelLo_Alarm	BOOL	–
TankLevel_LIT	REAL	–
TankEmpty_SP	REAL	2.0 ft
TankFull_SP	REAL	8.0 ft
TankLoAlarm_SP	REAL	1.0 ft
TankHiAlarm_SP	REAL	9.0 ft

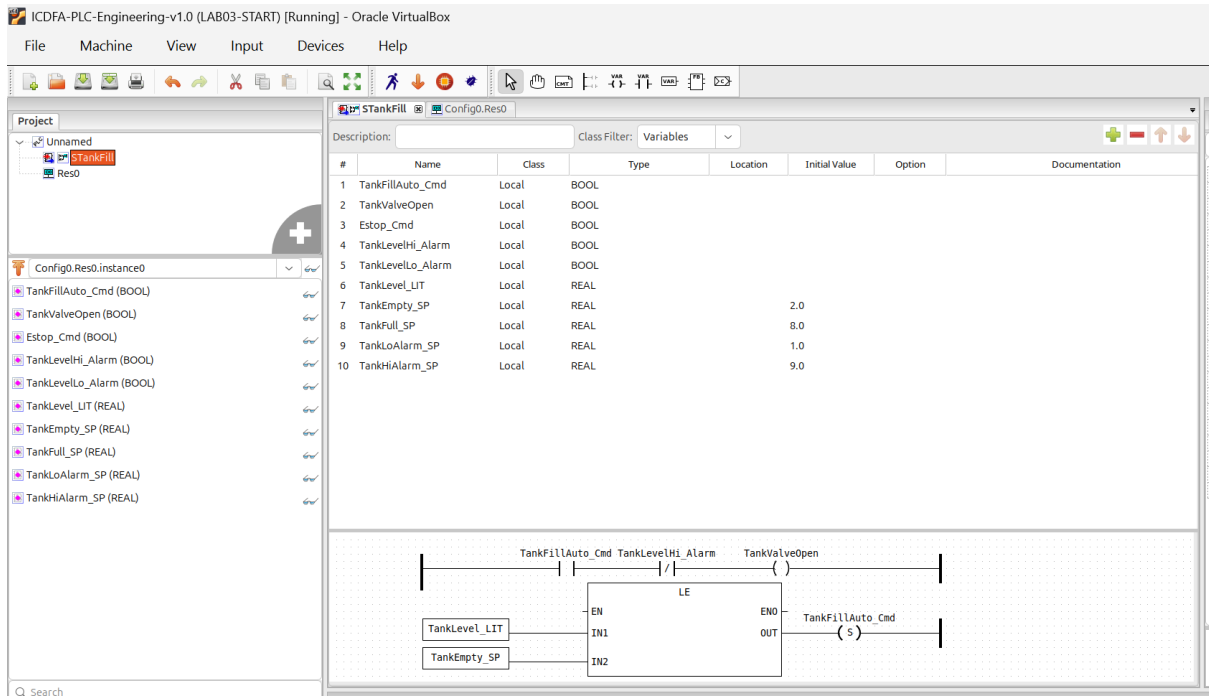


Figure 1: OpenPLC variable table showing the configured variables, data types and setpoint values.

3 Implemented Control Logic

The TankFill program implements the following control conditions:

- TankFillAuto_Cmd AND NOT TankLevelHi_Alarm energizes TankValveOpen.
- $\text{TankLevel_LIT} \leq \text{TankEmpty_SP}$ SETs TankFillAuto_Cmd.
- $\text{TankLevel_LIT} \geq \text{TankFull_SP}$ RESETs TankFillAuto_Cmd.
- Estop_Cmd RESETs TankFillAuto_Cmd.
- TankLevelHi_Alarm RESETs TankFillAuto_Cmd.
- $\text{TankLevel_LIT} \geq \text{TankHiAlarm_SP}$ energizes TankLevelHi_Alarm.
- $\text{TankLevel_LIT} \leq \text{TankLoAlarm_SP}$ energizes TankLevelLo_Alarm.

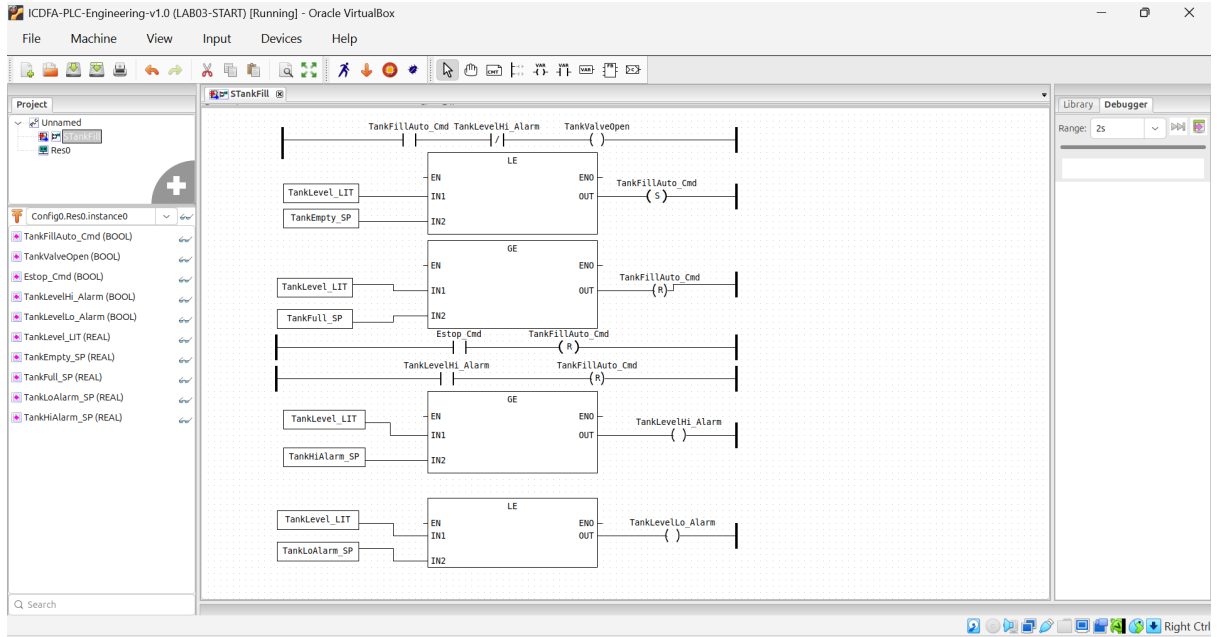


Figure 2: Completed TankFill ladder logic in OpenPLC Editor.

4 Build Validation

The comparison blocks initially generated compilation errors because their REAL input parameters were not connected.

The comparison inputs were corrected and connected as follows:

Table 3: Comparison block inputs

Block	IN1	IN2
LE	TankLevel_LIT	TankEmpty_SP
GE	TankLevel_LIT	TankFull_SP
GE	TankLevel_LIT	TankHiAlarm_SP
LE	TankLevel_LIT	TankLoAlarm_SP

After the correction, the program compiled successfully and generated `TankFill.so`.

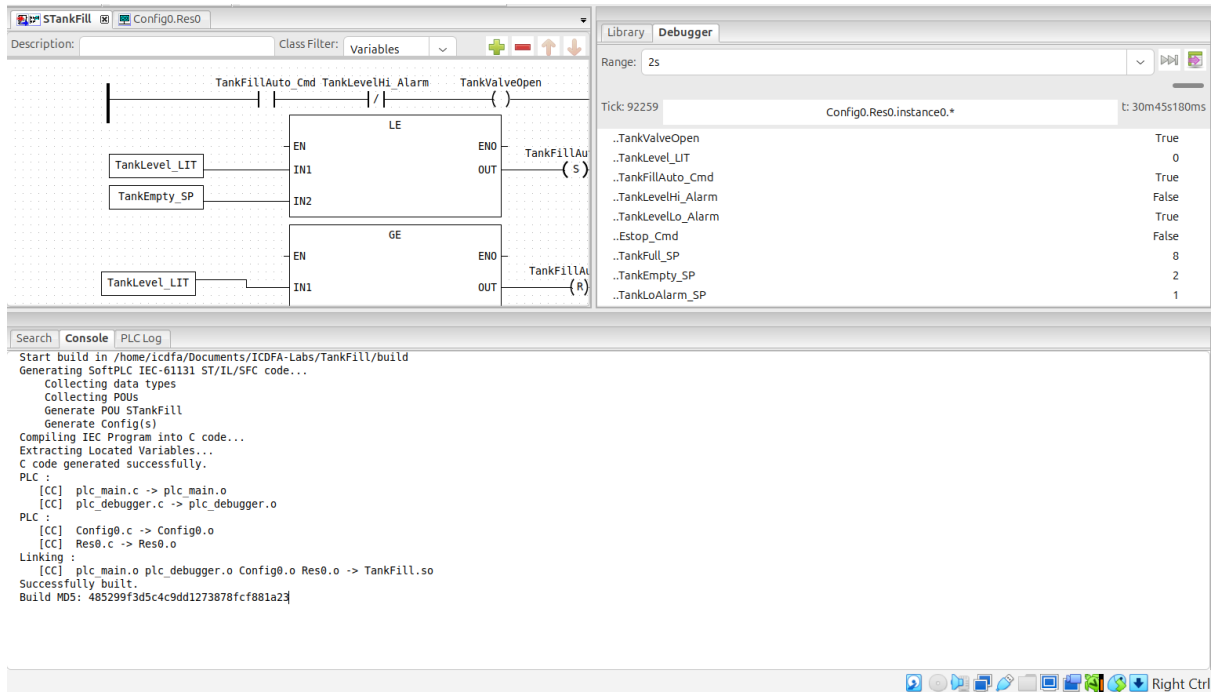


Figure 3: Successful OpenPLC build showing code generation and linking of TankFill.so.

5 PLC Runtime Validation

The generated program was successfully transferred to the PLC simulation and started.

The OpenPLC runtime reported:

PLC data transfered successfully.

PLC installed successfully.

PLCobject : PLC started

Starting PLC

The runtime also displayed cyclic-task timing warnings during simulation. These indicated that some simulated PLC cycles exceeded the configured interval while running inside the virtual machine.

6 Live Debugger Validation

The OpenPLC Debugger was used to monitor the PLC variables during execution.

For the observed empty-tank condition, the following values were recorded:

Table 4: Observed empty-tank condition

Variable	Observed Value
TankLevel_LIT	0
TankFillAuto_Cmd	TRUE
TankValveOpen	TRUE
TankLevelHi_Alarm	FALSE
TankLevelLo_Alarm	TRUE
Estop_Cmd	FALSE
TankFull_SP	8
TankEmpty_SP	2
TankLoAlarm_SP	1
TankHiAlarm_SP	9

The observed values are consistent with the configured logic. At 0 ft:

$$0 \leq 2.0$$

therefore the automatic-fill command is active.

Also:

$$0 \leq 1.0$$

therefore the low-level alarm is active.

Since the high-level alarm and emergency-stop command are FALSE, the fill valve remains TRUE.

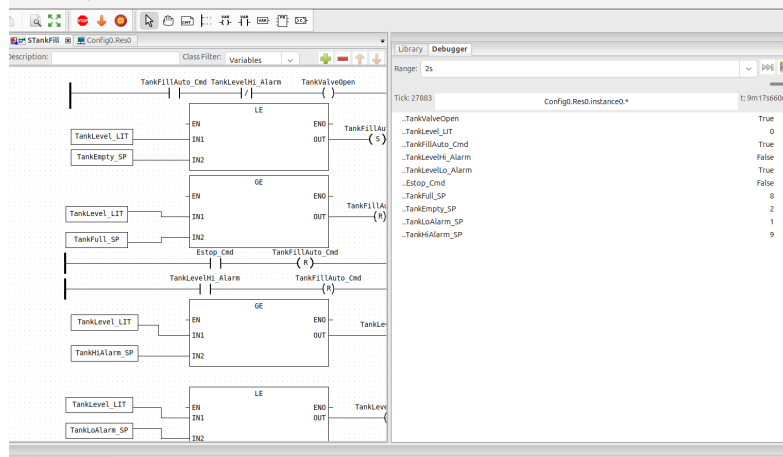


Figure 4: OpenPLC Debugger showing the observed empty-tank condition.

7 Logic Test Results

Table 5: Six-scenario TankFill logic test

Scenario	Condition	Expected Output
Empty tank	$\text{TankLevel_LIT} = 0 \text{ ft}$	$\text{TankFillAuto_Cmd} = \text{TRUE}$, $\text{TankValveOpen} = \text{TRUE}$ and $\text{TankLevelLo_Alarm} = \text{TRUE}$.
Normal filling	$2 \text{ ft} < \text{TankLevel_LIT} < 8 \text{ ft}$	Automatic filling remains active when no high-level alarm or emergency-stop condition is present.
Full tank	$\text{TankLevel_LIT} \geq 8 \text{ ft}$	TankFillAuto_Cmd is RESET and TankValveOpen becomes FALSE.
High alarm	$\text{TankLevel_LIT} \geq 9 \text{ ft}$	TankLevelHi_Alarm becomes TRUE and TankFillAuto_Cmd is RESET.
Low alarm	$\text{TankLevel_LIT} \leq 1 \text{ ft}$	TankLevelLo_Alarm becomes TRUE.
Emergency stop	$\text{Estop_Cmd} = \text{TRUE}$	TankFillAuto_Cmd is RESET and automatic filling stops.

The empty-tank condition was directly observed in the OpenPLC Debugger. The remaining conditions were verified against the implemented ladder logic and configured setpoints.

8 Safety Explanation

The emergency-stop condition must override normal automatic control because automatic filling should not continue when an emergency condition is active.

In the implemented program, **Estop_Cmd** RESETs **TankFillAuto_Cmd**. This removes the automatic-fill command and prevents normal automatic operation from continuing during an emergency stop.

9 Knowledge Check Responses

BOOL and REAL data types:

BOOL is appropriate for discrete commands, states and alarms because they use TRUE/FALSE values. REAL is appropriate for tank-level measurements and setpoints because they represent numerical values that can include decimal values.

SET/RESET pair:

The SET/RESET pair allows **TankFillAuto_Cmd** to remain latched after the automatic-start condition occurs and to be stopped when the tank reaches the full setpoint, an emergency stop occurs, or a high-level alarm becomes active.

Tank reaches 9 ft:

The high-level alarm becomes TRUE. **TankFillAuto_Cmd** is RESET, and the high-level alarm interlock prevents the fill valve from remaining open.

Simulation before physical equipment:

PLC logic should be validated in simulation to identify control, alarm, interlock and safety problems before connecting the program to physical industrial equipment.

10 Conclusion

The TankFill PLC program was successfully created and compiled in OpenPLC using Ladder Diagram. The required variables, setpoints, automatic fill logic, alarms and emergency stop logic were implemented.

The program was successfully built, transferred to the PLC simulation and started. The OpenPLC Debugger confirmed the expected behavior for the observed 0 ft tank condition.